This document gives you a **clean, working rewrite** of the Invoice system based on your Prisma models.

🎯What this solves

✔️Monthly invoices per **Cab Service** ✔️Click month → see **Vehicles → Trips → Cost breakdown** ✔️Draft = uninvoiced trips ✔️Pending / Paid = linked trips ✔️Invoice generation works ✔️Frontend gets **exact data it needs**

Core Design (IMPORTANT)

Invoice States

State	How it is calculated
Draft	`trip_costs.invoice_id = null`
Pending	Invoice exists + trips linked
Paid	Invoice exists + `status = Paid`

⚠️Draft invoices are **NOT stored** in DB.

API CONTRACT (FINAL)

Monthly Invoice List

```
GET /api/invoices?month=YYYY-MM
```

Returns grouped list per cab service.

Invoice Details (ONE ENDPOINT)

```
GET /api/invoices/service/:cabServiceId?month=YYYY-MM
```

Works for Draft / Pending / Paid.

Generate Invoice

```
POST /api/invoices/generate
```

🧩BACKEND IMPLEMENTATION

invoice.routes.ts

```
router.use(authenticate);

router.get('/', getMonthlyInvoices);
router.get('/service/:cabServiceId', getInvoiceDetailsByService);
router.post('/generate', generateInvoice);
router.post('/:id/pay', payInvoice);
```

---

invoice.controller.ts

```
export const getMonthlyInvoices = async (req, res) => {
  const { month } = req.query;
  const data = await invoiceService.getMonthlyInvoices(month);
  res.json(data);
};

export const getInvoiceDetailsByService = async (req, res) => {
  const { cabServiceId } = req.params;
  const { month } = req.query;

  const data = await invoiceService.getInvoiceDetails(cabServiceId, month);
  res.json(data);
};

export const generateInvoice = async (req, res) => {
  const result = await invoiceService.generateMonthlyInvoice({
    ...req.body,
    userId: req.user.id
  });
  res.status(201).json(result);
};
```

```
export const payInvoice = async (req, res) => {
  const result = await invoiceService.payInvoice(req.params.id, req.body);
  res.json(result);
};
```

---

invoice.service.ts (CORE LOGIC)

Monthly list (grouped)

```
export const getMonthlyInvoices = async (month: string) => {
  const services = await prisma.cab_services.findMany({
    include: { vehicles: true }
  });

  const result = [];

  for (const service of services) {
    const invoice = await prisma.invoice.findFirst({
      where: { cab_service_id: service.id, billing_month: month },
      orderBy: { created_at: 'desc' }
    });

    const trips = await prisma.trip_costs.findMany({
      where: {
        invoice_id: invoice ? invoice.id : null,
        trip_assignments: { vehicles: { cab_service_id: service.id } }
      }
    });

    if (!invoice && trips.length === 0) continue;

    result.push({
      cabServiceId: service.id,
      cabServiceName: service.name,
      month,
      status: invoice?.status || 'Draft',
      totalAmount: trips.reduce((s, t) => s + Number(t.total_cost), 0),
      invoiceId: invoice?.id || null
    });
  }

  return result;
};
```

Invoice Details (vehicles → trips)

```
export const getInvoiceDetails = async (cabServiceId: string, month: string) =>
{
  const invoice = await prisma.invoice.findFirst({
    where: { cab_service_id: cabServiceId, billing_month: month },
    orderBy: { created_at: 'desc' }
  });

  const trips = await prisma.trip_costs.findMany({
    where: {
      invoice_id: invoice ? invoice.id : null,
      trip_assignments: { vehicles: { cab_service_id: cabServiceId } }
    },
    include: {
      trip_assignments: {
        include: {
          vehicles: true,
          drivers: { include: { users_drivers_user_idTousers: true } }
        }
      }
    }
  });

  const vehicleMap = {};

  trips.forEach(t => {
    const v = t.trip_assignments.vehicles;
    if (!vehicleMap[v.id]) {
      vehicleMap[v.id] = {
        vehicleId: v.id,
        registrationNumber: v.registration_number,
        trips: [],
        totalCost: 0
      };
    }
    vehicleMap[v.id].trips.push(t);
    vehicleMap[v.id].totalCost += Number(t.total_cost);
  });

  return {
    cabServiceId,
    month,
    status: invoice?.status || 'Draft',
    totalAmount: Object.values(vehicleMap).reduce((s: any, v: any) => s +
v.totalCost, 0),
    breakdownByVehicle: Object.values(vehicleMap)
```

```
  };
};
```

Generate Invoice

```
export const generateMonthlyInvoice = async ({ cabServiceId, month, dueDate,
notes, userId }) => {
  const draft = await getInvoiceDetails(cabServiceId, month);

  if (draft.breakdownByVehicle.length === 0)
    throw new Error('No billable trips');

  const invoice = await prisma.invoice.create({
    data: {
      invoice_number: `INV-${Date.now()}`,
      cab_service_id: cabServiceId,
      billing_month: month,
      total_amount: draft.totalAmount,
      due_date: new Date(dueDate),
      status: 'Pending',
      notes,
      created_by: userId
    }
  });

  await prisma.trip_costs.updateMany({
    where: { invoice_id: null, trip_assignments: { vehicles: { cab_service_id:
cabServiceId } } },
    data: { invoice_id: invoice.id }
  });

  return invoice;
};
```

🏓 FRONTEND (Next.js / React)

Invoice List

```
useEffect(() => {
  fetch(`/api/invoices?month=${month}`).then(r => r.json()).then(setInvoices);
}, [month]);
```

Click Invoice

```
router.push(`/invoices/${cabServiceId}/${month}`);
```

Invoice Details Page

```
useEffect(() => {
  fetch(`/api/invoices/service/${cabServiceId}?month=${month}`)
    .then(r => r.json())
    .then(setInvoice);
}, []);
```

Render: - Vehicle card - Trips table - Cost breakdown

RESULT

✔️Monthly invoice visible ✔️Clicking shows vehicles ✔️Costs per trip ✔️Generate works ✔️No missing data

---

🖌️ Next Improvements

- PDF invoice export
- Tax summary per vehicle
- Overdue cron job

If you want, next I can: 1️⃣ Convert this into **ready-to-paste files** 2️⃣ Add **PDF export** 3️⃣ Improve performance with aggregation

---

STEP 1 — Backend Invoice Routes (NO /api)

`modules/invoice/invoice.routes.ts`

```
import { Router } from 'express';
import { getMonthlyInvoices, generateInvoice } from './invoice.controller.js';
import { authenticate } from '../../middleware/auth.js';

const router = Router();

// GET /invoices?month=2025-01
router.get('/', authenticate, getMonthlyInvoices);

// POST /invoices/generate
router.post('/generate', authenticate, generateInvoice);

export default router;
```

This matches your URL usage exactly: - `/invoices` - `/invoices/generate`

No `/api` anywhere.